TH Bingen Technical University of Applied Sciences

Department 2- Technology, Informatics and Economics

Applied Bioinformatics (B.Sc.)

# Protein Function Predictions Using Embedding and Convolutional Neural Networks (CNN)

Report in the subject Artificial Intelligence in Bioinformatics

By: Shatu Ahmed

Supervised by Dr. Asis Hallab

Matrikel-Nr: 2134985

28.02.2025

# Table of Contents

**Lists of abbreviations** ..... **III**

**1. Introduction** ..... **4**

**2. Materials and Methodes** ..... **5**

*2.1 Materials* ..... *5*

*2.2 Methods* ..... *6*

**3. Results** ..... **8**

**4. Discussion** ..... **13**

**5. References** ..... **14**

**6. List of Figures** ..... **15**

**7. List of Tables** ..... **16**

## Lists of abbreviations

MCC Mathew Correlation coefficient

CNN Convolutional Neural Networks

SGD Stochastic gradient descent

CV Cross validation

LSTM Long Short-Term Memory

# 1. Introduction

Understanding protein functions plays an essential role in the medical, biotechnology and biology fields. Current methods for determining these functions involve comparing the amino acid structure of a protein to a previously identified one that shares a similar structure (Alberts et al., 2014).

While experimental approaches including mutagenesis, mass spectrometry, and small-angle X-ray scattering (SAX) have proven to be expensive and time-consuming (Schnoes et al., 2009), machine learning-based ones have gained a rise in popularity in recent years (Min et al., 2017). Further exploration of these methods could therefore be beneficial for future protein classifications.

A method that has proven to be effective with deep-learning models is the use of vector representations of amino acids (Asgari and Mofrad, 2015).

Originally developed for statistical dimensionality reduction, Kidera factors encode the physicochemical properties of amino acids. Derived from principal component analysis, they can assist machine learning tasks in the form of embeddings (Kidera et al., 1985).

Convolutional Neural Networks (CNN) are described as one of the most effective deep learning architectures due to their spatial pattern recognition in structural data (Min et al., 2017). Combining such a model with physiochemical embeddings may prove more effective in analysing relevant protein functions compared to other models.

The present work investigates the effectiveness of training a CNN model to recognize protein functions when embedding their sequences using ten Kidera factors and distinguishing them by membership to a group of reference proteins.

# 2. Materials and Methodes

## 2.1 Materials

**Biological material**

- protein.fa : A fasta file containing referencing proteins used for classifying
- mapman.tsv: tsv file containing protein sequences and their assignments to groups (Mapman Bins) of the same molecular function
  Structure: two columns: BINCODE and IDENTIFIER
  The BINCODE represents the class/family/function of a protein and IDENTIFIER represents a protein identifier
- kidera_factors.csv: A CSV file containing ten Kidera factors that stem from the Rlang Peptide Package
  List of Kidera factors:
- Helix/bend preference
- Side-chain size
- Extended structure preference
- Hydrophobicity
- Double-bend preference
- Partial specific volume
- Flat extended preference
- Occurrence in alpha region
- pK-C
- Surrounding hydrophobicity

**Technical material:**

**Software**

- Python:
  Function: Programming language, used to carry out the specified tasks in this paper, including implementing the Embedding process and machine learning model
  Version: Python 3.11.7
- JupyterLab:
  Function: AI- server provided by TH Bingen, where all the scripts and program were carried out on due to the demanding RAM
  Version: 3.6.7
- https://app.diagrams.net
  Function: Flowchart maker and Diagram software
  Version: 26.0.16

A full list of all used python packages can be viewed in the GitLab repository.

**Hardware**

- Laptop:

Function: Device for implementing the model and evaluating the data as well writing the scientific report
Model: Lenovo IdeaPad 5 15ARE05
Version: Windows 11 Home 22H2

## 2.2 Methods

Before implementing the Embeddings step, the necessary data (Protein.fa and mapmap.tsv) was preprocessed by filtering the mapman.tsv by their largest bin per root bin. The bins, along with the identifiers, were extracted and merged with their corresponding protein sequence. This was done with a Python script (data_preprocessing.py). The result was 40,302 protein sequences, where each amino acid was then mapped to a vector of ten Kidera factors (Fig. 1). These sequences stem from the new updated dataset.

The subsequent Nx10 matrix (N, representing the sequence length) was padded and truncated to varying fixed lengths (e.g., 1500) and transformed into a three-dimensional array that contained the dimensions: batch size, which includes the number of sequences that would enter the CNN model for training, sequence length, and number of features (ten Kidera factors.).

The implementation of the CNN was done with the script (labeling_training.py) and consists of four convolutional layers, each followed by a max pooling layer to reduce spatial dimensions (Fig 2). This works by dividing the input data into smaller regions and selecting the maximum value within each fixed region (Zhao and Zhang, 2024). After the fourth pooling layer, an additional flattening layer is applied that leads to a fully connected layer. Lastly, a Sigmoid activation function is applied to the model. Since this task requires multi-label classification, the sigmoid function was selected to allow independent probability estimation for each protein classification bin. The output of sigmoid functions exit between the ranges of 0 and 1, thereby operating well for binary classification (Ali, 2024).

Binary-Cross-Entropy is the loss function typically used for binary classification and was selected here with the intent of reducing loss between target and predicted outputs by comparing them (Yathish, 2025).

To reduce noise in the training set, different optimization methods, including Adam and stochastic gradient descent (SGD), were selected and tested along with varying learning rates (Table 1). Training of the model involved applying a random split of 80:20 on the input data, meaning 80% of the sequences were used for training and 20% for validation.

Lastly, a cross-validation (CV) function was implemented to confirm the efficiency of the results. For CV, the test dataset was split into 5 subsets, called k folds (k = 5). These folds are then looped, four of them being used for training and the remaining data fold being used for validation. The estimated score, derived from these iterations, is the average of all values being computed (Scikit-Learn Developers, 2025, February 27).

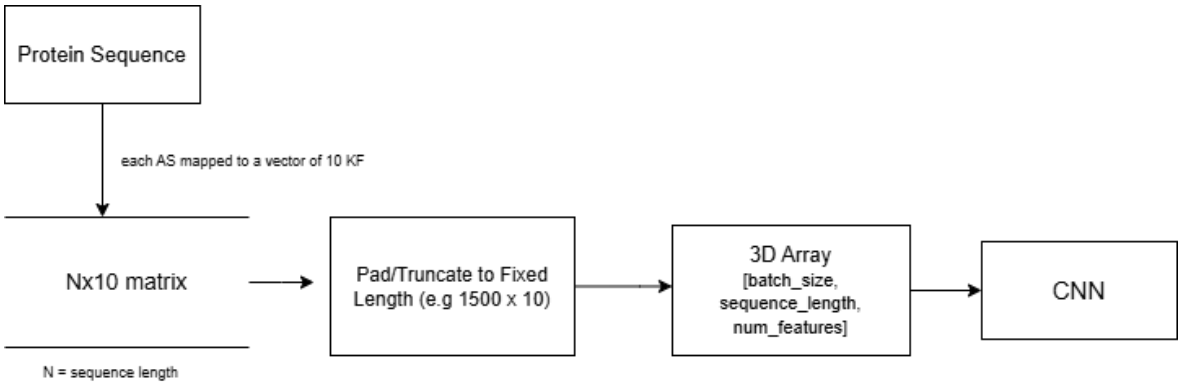


*Figure 1: A flowchart, visualizing the Embedding process before the data is used for traing in the CNN model. AS describes the amino acids in a sequence, which are being embedded to 10 Kidera factors. The resulting vector is padded or truncated and transformed into a 3D array.*

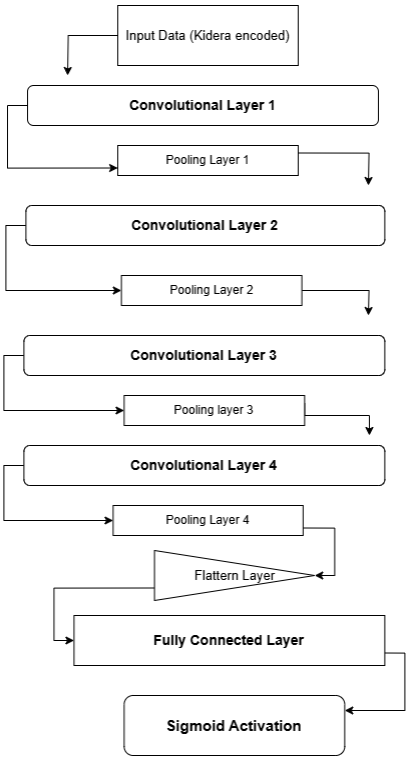


*Figure 2: Sketch of the CNN architecture, including max pooling layers applied after each convolutional layer*

## 3. Results

A sequence length of 1500 and a learning rate of 0.0001 lead to in a Model with a Mathew Correlation coefficient of 0.9074 (Fig. 3a). The activation function Adam was used for this simulation. Training stops at approximately 20 epochs, as can be seen through the steady fall of the blue graph. From epoch 12, the graph of validation loss starts to go above the training loss graph, indicating overfitting. Every simulation, performed with a batch size of 1500, can be viewed in Table 2.

When applying a weight decay of 1E-4, precision and MCC score slightly decrease, yet remain above 0.95. Simulations, which used SGD as an optimizer, displayed lower statistic scores. Applying Weight decay and dropout layers to Adam did not increase the statistical scores.

When increasing the sequence length to 3500 and applying a learning rate of 0.0001 with the optimizer Adam (Fig 3b), the model runs similarly to the one in Fig 3a. The training loss graph appears identical to the graph depicted in Fig. 3a, whereas the validation loss graph (Fig. 3b) runs slightly higher towards a height of 0.3, exhibiting more bends than Fig. 3a.

Increasing the sequence length resulted in lower statistical scores compared to a length of 1500, with the same parameters. This can be viewed in Table 3.

Models with SGD and a learning rate of 0.1 can be seen in Fig. 4a and b.

Both models perform similarly, in the way their two graphs (validation and Training) fall almost identical. From epoch 2, the graphs fall rapidly until epoch 5, where it starts slowing down. The graph for validation loss starts to rise above training after epoch 20, with Fig 4a exhibiting more bends towards the end of its run compared to Fig 4b. Their statistical scores display similar results, with Fig. 4a demonstrating slightly higher numbers.

Fig. 5 displays the Cross-Validation results of the graph in Fig. 3a. Compared to every other simulation, a sequence length of 1500 with an Adam optimizer demonstrates the highest numerical scores, with an average MCC score of 0.9690.

Figure 6 showcases a plot with the old dataset, which was not further used for this training, based on the low statistical scores.

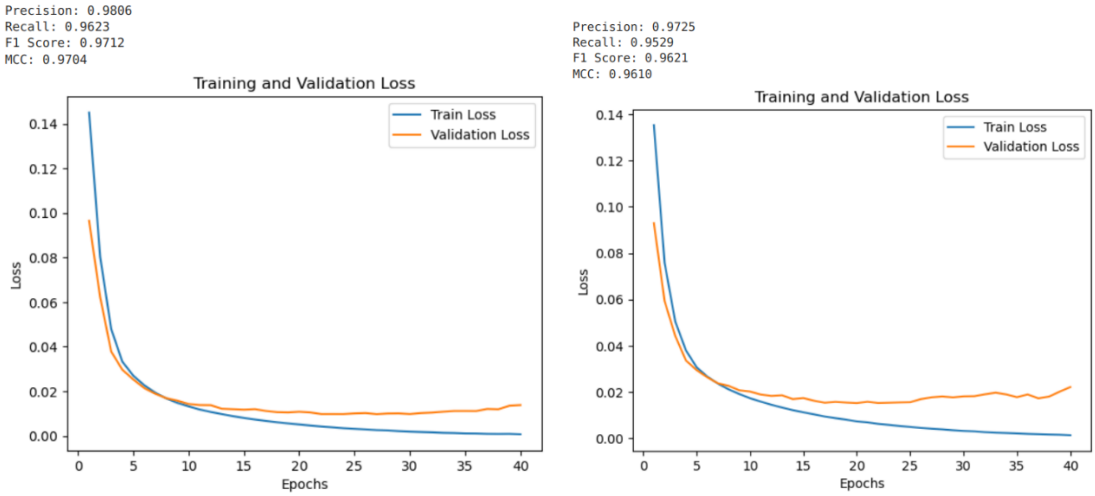


*Figure 3: Results of the CNN training for both a sequence length of 1500 and 3500. Depicted are training and validation loss curves using the Adam optimizer with a learning rate of 0.0001. The increasing gap between validation and training loss indicates potential overfitting. Statistical scores are depicted above each plot. All scores are above 0.95 which shows high accuracy and precision.*

*(a) Left side, displays the graph with a sequence length of 1500*

*(b) Right side, displays the graph with a sequence length of 3500*

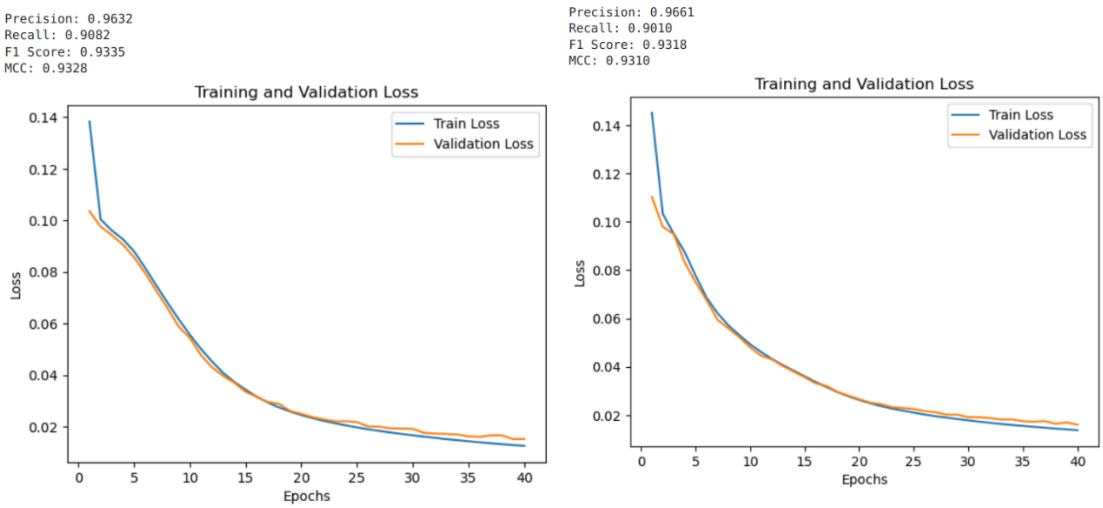


*Figure 4: Results of the CNN training for both a sequence length of 1500 and 3500. Depicted are training and validation loss curves using the SGD optimizer with a learning rate of 0.1 and a weight decay of $10^{-4}$*

*(a) Left side, displays the graph with a sequence length of 1500*
*(b) Right side, displays the graph with a sequence length of 3500*

```
Fold 1/5
Fold 1 - Train Loss: 0.0012, Val Loss: 0.0135
Precision: 0.9767, Recall: 0.9606, F1: 0.9684, MCC: 0.9675
Fold 2/5
Fold 2 - Train Loss: 0.0006, Val Loss: 0.0150
Precision: 0.9796, Recall: 0.9625, F1: 0.9707, MCC: 0.9699
Fold 3/5
Fold 3 - Train Loss: 0.0010, Val Loss: 0.0105
Precision: 0.9810, Recall: 0.9614, F1: 0.9708, MCC: 0.9701
Fold 4/5
Fold 4 - Train Loss: 0.0015, Val Loss: 0.0121
Precision: 0.9820, Recall: 0.9603, F1: 0.9709, MCC: 0.9700
Fold 5/5
Fold 5 - Train Loss: 0.0010, Val Loss: 0.0128
Precision: 0.9792, Recall: 0.9581, F1: 0.9684, MCC: 0.9675

Cross-Validation Results:
Average Train Loss: 0.0011
Average Val Loss: 0.0128
Average Precision: 0.9797
Average Recall: 0.9606
Average F1 Score: 0.9698
Average MCC: 0.9690
```

*Figure 5: Cross-Validation Results that were performed on the Model with a sequence length of 1500, Adam optimizer and a learning rate of 0.0001. Five folds were used, with 4 of these being used for training in each iteration and being validated with remaining fold. The results represent the average statistical results calculated in each fold.*

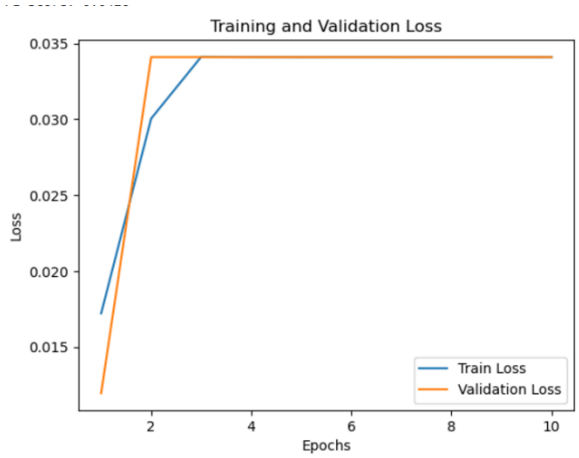


*Figure 6: Results of a CNN model performed with the old dataset, as described in the GitLab repository*

Table 1: CNN- Model parameters, displaying the number of layers, input features and other aspects of the architecture.

| **Layer** | **Input feature maps** | **Output feature maps** | **Kernel size** | **Stride** |
|---|---|---|---|---|
| 1st | 10 | 32 | 7 | 2 |
| 2nd | 32 | 64 | 5 | 2 |
| 3rd | 64 | 128 | 3 | 2 |
| 4th | 128 | 256 | 3 | 2 |

Table 2: Overview of simulations and their scores performed on a sequence length of 1500, max pooling, different optimizers (Adam, SGD) and varying learning rates. DL 1-5 stand for dropout layers which were applied after pooling. DL fc was applied after the last fully connecting layer of the CNN model.

| **Epoch Size** | **Optimizer** | **Max Pooling** | **Learning Rate** | **Weight Decay** | **DL 1** | **DL 2** | **DL 3** | **DL 4** | **DL fc** | **Precision** | **Recall** | **F1-Score** | **MCC** |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 40 | Adam | Yes | 0.0001 | None | None | None | None | None | None | 0.9806 | 0.9623 | 0.9712 | 0.9704 |
| 40 | Adam | Yes | 0.0001 | 1,00E-04 | None | None | None | None | None | 0.9782 | 0.9534 | 0.9655 | 0.9645 |
| 40 | SGD | Yes | 0.1 | 1,00E-04 | None | None | None | None | None | 0.9632 | 0.9082 | 0.9335 | 0.9328 |
| 40 | SGD | Yes | 0.01 | 1,00E-04 | None | None | None | None | None | 0.4763 | 0.1764 | 0.2287 | 0.3488 |
| 40 | Adam | Yes | 0.0001 | 1,00E-04 | 0.2 | 0.2 | 0.2 | 0.2 | 0.5 | 0.9825 | 0.9369 | 0.9585 | 0.9581 |
| 40 | Adam | Yes | 0.0001 | 1,00E-04 | None | None | 0.2 | 0.2 | 0.5 | 0.9848 | 0.9406 | 0.9619 | 0.9612 |

Table 3: Overview of simulations and their scores performed on a sequence length of 3500 with max pooling, different optimizers (Adam, SGD) and varying learning rates.

| **Epoch Size** | **Optimizer** | **Max Pooling** | **Learning Rate** | **Weight Decay** | **Precision** | **Recall** | **F1-score** | **MCC** |
|---|---|---|---|---|---|---|---|---|
| 40 | Adam | Yes | 0.0001 | None | 0.9725 | 0.9529 | 0.9621 | 0.9610 |
| 40 | Adam | Yes | 0.0001 | 1,00E-04 | 0.9739 | 0.9444 | 0.9586 | 0.9575 |
| 40 | SGD | Yes | 0.1 | 1,00E-04 | 0.9661 | 0.9010 | 0.9318 | 0.9310 |
| 40 | Adam | Yes | 0.00001 | None | 0.9128 | 0.7698 | 0.8300 | 0.8399 |

Table 4: Display of models with varying optimizers including their sequence length and the time it took the model to train.

| Optimizer | Sequence Length | Time in minutes |
|---|---|---|
| Adam | 1500 | 15.77 |
| SGD | 1500 | 17.34 |
| Adam | 3500 | 30.70 |

## 4. Discussion

We investigated the recognition of protein functions of unknown proteins using a CNN model trained on reference protein sequences, which were classified into bins based on their functional categories. The training data itself was embedded using ten Kidera factors.

A variety of optimizers, loss functions, sequence lengths and learning rates were tested to evaluate their performance and efficiency. Results were assessed based on their F-score, precision, and MCC.

Overall, models with the Adam optimizer performed more efficiently based on their scores (Fig. 3ab). The Model with a sequence length of 1500 had the highest MCC score, taking all elements of a confusion matrix into account (False Positives, True Positives, False Negatives, and True Negatives). The closer it is to 1+, the more accurate the predictions are (KoshurAI, 2025, February 27). The CV results in Figure 5 also demonstrate an efficient working model, grounded on its high prediction scores and low loss assessments. It also took less time to execute, based on the measures in Table 4. Nevertheless, there is a minor overfitting issue, indicated by the graph of validation loss slightly stagnating above training loss (Fig. 3ab).

Using SGD as an optimizer demonstrates no substantial gap between validation and training loss, indicating a lesser degree of overfitting (Fig 4ab). All scores are slightly lower in comparison to Adam, especially the Recall value, meaning the model is missing more True Positive cases. While SGD may appear more stable, it is less sensitive and takes longer to compute (Table 4).

It is worth mentioning that this model was trained on a filtered dataset with 40,302. The non-filtered dataset includes 102,0705 and requires a higher CPU, even GPU, performance, which could not be reproduced due to time constraints. With the adequate hardware, further exploration of these models with the larger dataset should be considered.

From the difference in results between the new filtered dataset and the old dataset (Fig. 6), the necessity of high-quality data becomes more apparent. Thus, the old dataset was abandoned as it provided less efficient results.

In addition to data quality, the architecture itself should also be prioritized. Gallego et al. (2018) suggest replacing the max pooling layer with a k-Nearest Neighbour (kNN) algorithm. This method seems to slightly outperform other CNN architectures when applied to noisy datasets. Another method would be the combination of CNN with Long Short-Term Memory (LSTM), as demonstrated by Rao and Reddy (2020). This method showed higher classification results than traditional architectures. A hybrid model approach would thus be an interesting to research further.

The CNN model provided efficient results and can be considered successful in predicting protein functions, even in comparison to the other models, which were conducted by different groups (RNN-Model, Transformer-Model). The statistical scores are slightly lower in comparison but take less time to train. Therefore, the CNN-experiment should be considered for more thorough research and exploration.

## 5. References


Ali, M. (2024). Introduction to activation functions in neural networks. Retrieved from https://www.datacamp.com/tutorial/introduction-to-activation-functions-in-neural-networks

Asgari, E., & Mofrad, M. R. (2015). Continuous distributed representation of biological sequences for deep proteomics and genomics. PLOS ONE, 10(11). https://doi.org/10.1371/journal.pone.0141287

Alberts, B., Johnson, A., Lewis, J., Raff, M., Roberts, K., & Walter, P. (2014). *Molecular biology of the cell* (6th ed.). Garland Science.

Gallego, A.-J., Pertusa, A., & Calvo-Zaragoza, J. (2018). Improving convolutional neural networks' accuracy in noisy environments using k-nearest neighbors. *Applied Sciences*, 8(11), 2086. https://doi.org/10.3390/app8112086

Kidera, A., Konishi, Y., Oka, M., Ooi, T., & Scheraga, H. A. (1985). Statistical analysis of the physical properties of the 20 naturally occurring amino acids. *Journal of Protein Chemistry*, 4(1), 23–55. https://doi.org/10.1007/BF01025492

KoshurAI. (2025, February 27). Understanding the Matthews correlation coefficient (MCC) in machine learning. Retrieved from https://koshurai.medium.com/understanding-the-matthews-correlation-coefficient-mcc-in-machine-learning-26e8049f8572

Min, S., Lee, B., & Yoon, S. (2017). Deep learning in bioinformatics. *Briefings in Bioinformatics*, 18(5), 851–869. https://doi.org/10.1093/bib/bbw068

Rao, M. S., & Reddy, B. E. (2020). An improved convolutional neural network with LSTM approach for texture classification. *International Journal*, 8(7).

Schnoes, A. M., Ream, D. C., Thorman, A. W., Babbitt, P. C., & Friedberg, I. (2013). PLoS Computational Biology, 9(5). https://doi.org/10.1371/journal.pcbi.1003063

Scikit-Learn Developers. (2025, February 27). 3.1. Cross-validation: Evaluating estimator performance. Retrieved from https://scikit-learn.org/stable/modules/cross_v

Yathish, V. (2025, February 27). Loss functions and their use in neural networks. Retrieved from https://towardsdatascience.com/loss-functions-and-their-use-in-neural-networks-a470e703f1e9/

Zhao, L., & Zhang, Z. (2024). An improved pooling method for convolutional neural networks. Scientific Reports, 14, 1589. https://doi.org/10.1038/s41598-024-51258-6

Gitlab repository:
Beketova, A & Ahmed, S. (2025, February 28). *Embedding-Approach* (v1.0). GitLab. https://gitlab.rlp.net/a.hallab/aibi-ws-2024-25/-/tree/Embedding-Approach?ref_type=heads

## 6. List of Figures

Figure 1: A flowchart, visualizing the Embedding process before the data is used for traing in the CNN model. AS describes the amino acids in a sequence, which are being embedded to 10 Kidera factors. The resulting vector is padded or truncated and transformed into a 3D array. .....7
Figure 2: Sketch of the CNN architecture, including max pooling layers applied after each convolutional layer.....7
Figure 3: Results of the CNN training for both a sequence length of 1500 and 3500. Depicted are training and validation loss curves using the Adam optimizer with a learning rate of 0.0001. The increasing gap between validation and training loss indicates potential overfitting. Statistical scores are depicted above each plot. All scores are above 0.95 which shows high accuracy and precision.....9
Figure 4: Results of the CNN training for both a sequence length of 1500 and 3500. Depicted are training and validation loss curves using the SGD optimizer with a learning rate of 0.1 and a weight decay of $10^{-4}$.....9
Figure 5: Cross-Validation Results that were performed on the Model with a sequence length of 1500, Adam optimizer and a learning rate of 0.0001. Five folds were used, with 4 of these being used for training in each iteration and being validated with remaining fold. The results represent the average statistical results calculated in each fold.....10
Figure 6: Results of a CNN model performed with the old dataset, as described in the GitLab repository.....10

# 7. List of Tables

Table 1: .....11
Table 2: .....11
Table 3: .....12
Table 4: .....12

## Erklärung zur Originalität der Arbeit

Hiermit bestätige ich, dass die abgegebene Arbeit das Original ist und von mir ohne weitere Hilfe geschrieben wurde. Wenn Arbeit anderer referenziert oder genutzt wurde, wurde dies angemessen kenntlich gemacht. Meine Arbeit wurde noch nicht bewertet oder veröffentlicht. Die elektronisch abgegebene Version stimmt mit der abgegebenen überein

.....

Bingen, 28.02.2025

## Erklärung zu Eigentum und Urheberrecht

Hiermit erkläre ich mein Einverständnis, dass die Technische Hochschule Bingen, Fachbereich 2, die vorliegende wissenschaftliche Arbeit den Studierenden und interessierten Dritten zur Einsichtnahme zur Verfügung stellt und unter Nennung meines Namens (Shatu Ahmed) veröffentlichen darf.

.....

Bingen, 28.02.2025